

Tasca S4.01. BigQuery Avançat & Analytics Engineering

Requisits de compleció

Per a fer: Fer una tramesa

Per a fer: Rebre una qualificació

Descripció

Com a Analytics Engineer, la teva responsabilitat no és només lliurar dades, sinó assegurar l'escalabilitat i l'eficiència de la plataforma.

En aquest sprint final, redissenyaràs l'arquitectura de les dades per optimitzar costos (**Partitioning**), introduiràs lògica analítica avançada (**Window Functions**) i modelaràs estructures complexes (**Arrays**) per tancar el cicle amb l'automatització.

⚠ Important

- » **Control de Costos:** A BigQuery, cada consulta costa diners. Abans d'executar res, fixa't sempre en el missatge del validador ("Dry Run") a dalt a la dreta.
- » **Ubicació de les Dades:** Assegura't que el teu Dataset estigui a la regió EU (Multiple regions) o les connexions fallaran.

Limitacions del Sandbox

La zona de proves de BigQuery (Sandbox) està subjecta als següents límits:

- » S'apliquen tots els límits i quotes habituals de BigQuery.
- » Tens 10 GB d'emmagatzematge actiu i 1 TB de dades de consulta processades cada mes sense cap cost.
- » ⚠ **Caducitat:** Totes les taules, vistes i particions caduquen de forma automàtica després de **60 dies**.
- » No admet funcions avançades com: Transmissió de dades (Streaming), DML (UPDATE/DELETE massius) o el Servei de transferència de dades.

Nivell 1: Entorn i Ingesta Híbrida (Code-First)

Escenari:

El quadre de comandament financer tarda massa a carregar i el cost és inacceptablement alt. S'ha identificat que el problema crític són els informes que creuen vendes amb regions geogràfiques.

Exercici 1: Consulta sobre Taula no Optimitzada (Diagnòstic)

El Country Manager d'Alemanya necessita revisar urgentment les transaccions del dia **12 de març de 2022**.

Tasca:

- » 1. Escriu la consulta que uneix (JOIN) transaccions i companyies.
- » 2. Filtra els resultats per la data indicada i el país "Germany".
- » 3. Sense executar la consulta, realitza un "Dry Run" (auditoria de costos).

```
17
18
19 SELECT * FROM `sprint3_silver.transactions_clean` as t
20 JOIN `sprint3_silver.companies_clean` as c
21 ON t.company_id = c.company_id
22 WHERE t.timestamp = '2022-03-12'
23     AND country = 'Germany'
```

✓ Esta consulta procesará 11.86 MB cuando se ejecute.

Exercici 2: Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster)

Escenari:

Després del diagnòstic de l'exercici anterior, hem confirmat que filtrar la taula de transaccions per data o per empresa provoca un *Full Table Scan* (lectura completa de la taula), cosa que és ineficient. Per solucionar-ho a la capa de consum (*Gold*), materialitzarem una versió optimitzada de la taula de fets.

Problema del sandbox: com que utilitzem un entorn gratuït sense targeta de crèdit, BigQuery esborra automàticament les dades de les particions amb més de 60 dies d'antiguitat. Atès que les nostres dades històriques són antigues, si particionem directament la taula, aquesta acabarà quedant buida. Per evitar-ho, ho resoldrem en dos passos: primer, actualitzarem automàticament les dates de les dades i, després, aplicarem l'optimització física.

Tasca:

Pas 1: Generació de Dades Recents (Mocking Data)

Crea una taula intermèdia anomenada `sprint3_silver.transactions_recent` a partir de la taula `sprint3_silver.transactions_clean`. El teu objectiu és mantenir totes les columnes, però substituir el timestamp original per un de nou, generat aleatòriament perquè caigui dins dels últims 50 dies.

» **Pista Tècnica:** Per fer-ho sense escriure totes les columnes a mà, pots utilitzar `SELECT * EXCEPT(timestamp)`. Per calcular la data aleatòria, hauràs de restar una quantitat de dies a l'instant actual.

Et seran molt útils les funcions `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`,

i la combinació `CAST(RAND() * 50 AS INT64)` per generar els dies de desfasament.

`TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST(RAND() * 50 AS INT64) DAY)`

```
1 CREATE OR REPLACE TABLE `sprint3_silver.transactions_recent`
2 AS SELECT * EXCEPT(timestamp),
3   TIMESTAMP_SUB(
4     CURRENT_TIMESTAMP(),
5     INTERVAL CAST(RAND() * 50 AS INT64) DAY
6   ) as timestamp
7 FROM `sprint3_silver.transactions_clean`
8
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Recientes

IK

▼ 9:27 p.m.

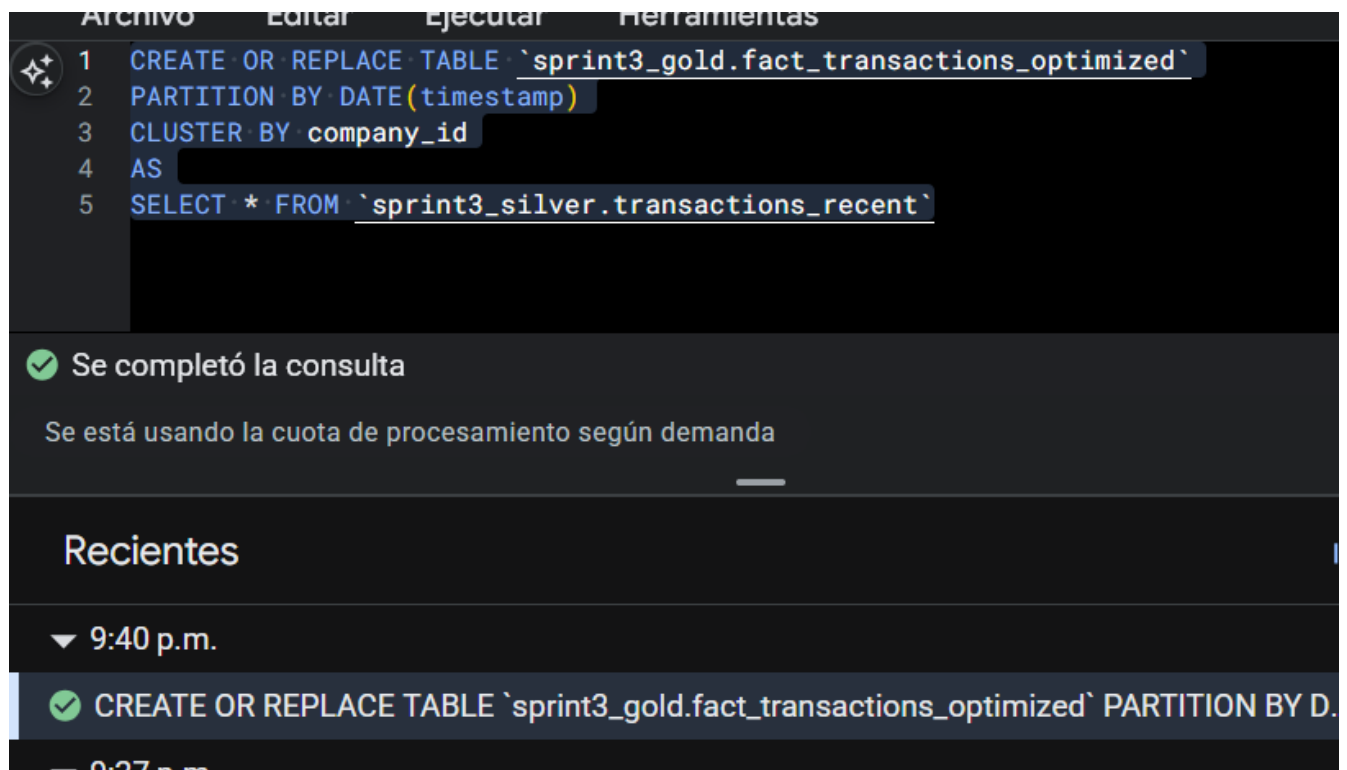
✓ CREATE OR REPLACE TABLE `sprint3\_silver.transactions\_recent` AS SELECT * EXCEPT(ti...

Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering)

Ara, crea la teva taula definitiva sprint3_gold.fact_transactions_optimized a partir de les dades recents que acabes de generar a transactions_recent. Has de construir la sentència DDL per configurar la taula amb les següents estratègies físiques d'emmagatzematge:

- » **Particionament (Partitioning)**: Divideix la taula per la data del camp DATE(timestamp). Això permetrà que les consultes que filtrin per dia (WHERE date = ...) només llegeixin la partició necessària. `PARTITION BY DATE(timestamp)`
- » **Clustering**: Ordena les dades dins de cada partició per business_id. Això accelerarà dràsticament els filtres per companyia i els encreuaments (JOINS) amb la dimensió d'empreses.

business_id -> lo remembre a company_id
`CLUSTER BY company_id`



```
1 CREATE OR REPLACE TABLE `sprint3_gold.fact_transactions_optimized`
2 PARTITION BY DATE(timestamp)
3 CLUSTER BY company_id
4 AS
5 SELECT * FROM `sprint3_silver.transactions_recent`
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Recientes

▼ 9:40 p.m.

✓ CREATE OR REPLACE TABLE `sprint3\_gold.fact\_transactions\_optimized` PARTITION BY D.

9:27 p.m.

👉 Entrega:

- » El codi SQL utilitzat per al Pas 1.
- » El codi DDL (CREATE OR REPLACE TABLE...) utilitzat al Pas 2.
- » Una captura de pantalla dels "Detalls" de la nova taula fact_transactions_optimized on es pugui comprovar que està particionada i agrupada en clústers correctament.

sprint3-analytics-eduvallejo / Datasets / sprint3_gold / Tables / fact_transactions_optimized	
☆ fact_transacti...	Consulta Chat Abrir en ▾ Compartir ▾ Copiar
Esquema	Detalles Vista previa Estadísticas Linaje Perfil de datos Calidad de los datos
Etiquetas	
Claves primarias	
Etiquetas	
Tipo de tabla	Particionada
Particionada por	DAY
Particionada en el campo	timestamp
Vencimiento de la partición	60 días
Filtro de partición	No es obligatorio
Agrupada por	company_id

Exercici 3: La Prova del Cotó (Benchmark)

Escenari:

Escenari: Ja has creat la taula `sprint3_gold.fact_transactions_optimized` amb les estratègies de Particionament i Clustering.

Ara toca demostrar a l'equip la millora de rendiment i l'estalvi de costos que has aconseguit. Simularem una petició típica de negoci: "Volem veure totes les transaccions dels últims 30 dies".

🔧 Tasca:

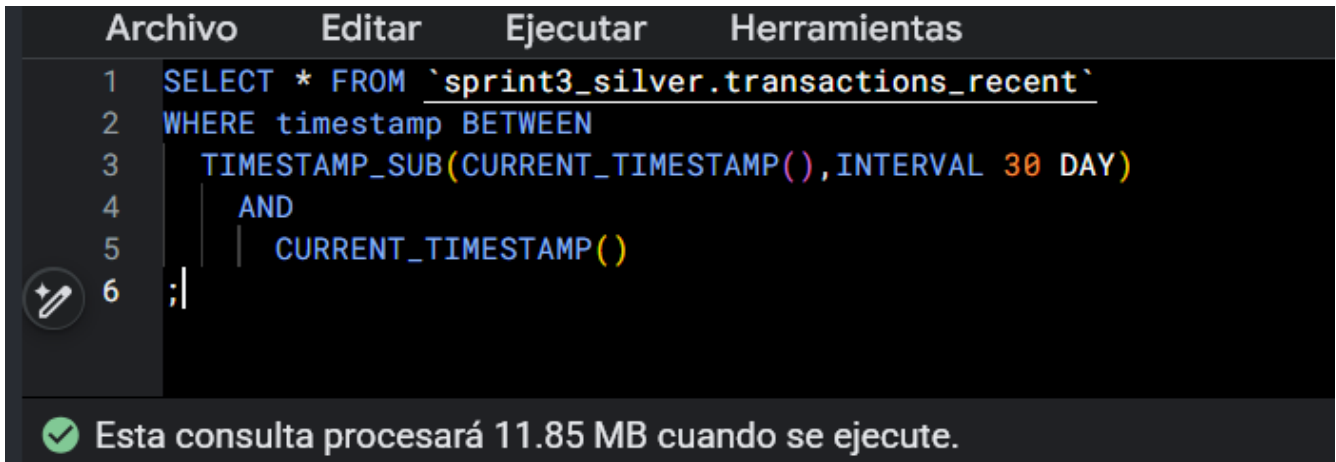
L'objectiu és clar i directe: **aplicar exactament la mateixa consulta a dues taules diferents i comparar-ne el cost computacional.**

Construeix una consulta SQL que seleccioni totes les columnes (`SELECT *`) i filtri les dades dels últims 30 dies.

⚠️ **NO executis les consultes encara.** En aquest exercici només utilitzarem el validador de BigQuery ("Dry Run", a dalt a la dreta de l'editor) per visualitzar els **Bytes processats** de manera gratuïta.

Fes la comparativa seguint aquests dos passos:

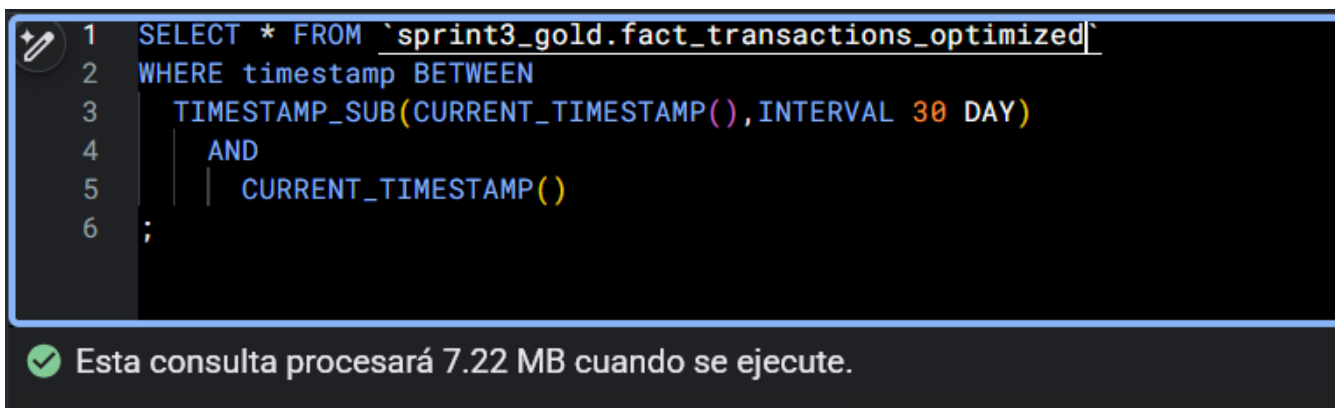
- » **Pas 1 (Taula no optimitzada):** Apunta la teva consulta a la taula `sprint3_silver.transactions_recent`. Observa el validador i fes una captura de pantalla dels bytes que processaria.



```
1 SELECT * FROM `sprint3_silver.transactions_recent`
2 WHERE timestamp BETWEEN
3     TIMESTAMP_SUB(CURRENT_TIMESTAMP(),INTERVAL 30 DAY)
4     AND
5     CURRENT_TIMESTAMP()
6 ;
```

✓ Esta consulta procesará 11.85 MB cuando se ejecute.

- » **Pas 2 (Taula optimitzada):** Sense canviar res de la lògica de la teva consulta, modifica només el FROM perquè ara apunti a la taula particionada `sprint3_gold.fact_transactions_optimized`. Observa com canvia el validador i fes la segona captura.



```
1 SELECT * FROM `sprint3_gold.fact_transactions_optimized`
2 WHERE timestamp BETWEEN
3     TIMESTAMP_SUB(CURRENT_TIMESTAMP(),INTERVAL 30 DAY)
4     AND
5     CURRENT_TIMESTAMP()
6 ;
```

✓ Esta consulta procesará 7.22 MB cuando se ejecute.

👉 Entrega:

- » El codi SQL base que has dissenyat.
- » Les dues captures de pantalla on es vegi la diferència de "Bytes processats" al validador.
- » Un breu text calculant quin és el % d'estalvi que has aconseguit (compara els bytes del Pas 1 respecte als del Pas 2).

La tabla optimizada consume solo el 60% de la tabla sin optimizar, te ahorras un 40%

Exercici 4: Smart Caching (Vistes Materialitzades)

Escenari:

El Director General té un quadre de comandament que mostra les "**Vendes Totals per Dia**".
Ell (i 50 managers més) refresquen aquest gràfic constantment.

✂ Tasca:

- » Crea una **Vista Materialitzada** anomenada `sprint3_gold.mv_daily_sales` que mostri les **Vendes Totals per Dia**.

👉 Entrega:

- » Codi per Crear la **Vista Materialitzada**.

```
1 CREATE MATERIALIZED VIEW IF NOT EXISTS `sprint3_gold.mv_daily_sales`
2 AS
3 SELECT DATE(timestamp) as day, SUM(amount) as total_sales
4 FROM `sprint3_gold.fact_transactions_optimized`
5 WHERE declined = 0
6 GROUP BY day;
```

✓ Esta consulta procesará 0 B cuando se ejecute.

Recientes	Resultados de
▼ 10:43 a.m.	< Información
✓ CREATE MATERIALIZED VIEW IF NOT EXISTS `sprint3_g...	Esta decla

- » Fes una consulta que mostri la vista creada i fes una captura de pantalla que mostri en el seu conjunt la vista materialitzada creada, amb la consulta executada i els bits processats.

Nivell 2: SQL Analític Avançat

Exercici 1: Perfilat de Clients VIP (Mètriques Agregades amb CTEs)

✂ Tasca:

- » 1. Crea una CTE anomenada **VIP_Stats** que agrupi per usuari i calculi:
 - La Despesa Total (SUM).
 - La Quantitat de Transaccions (COUNT).
 - El Tiquet Mitjà (AVG), arrodonit a 2 decimals.
 - La Compra Màxima (MAX).
 - Filtre: Mantén només aquells la Despesa Total dels quals sigui > 500.
- » Creua la CTE amb **users_combined** per obtenir les dades personals.

Requisits de Sortida:

- Columnes: **user_id**, **nom_complet**, **email**, **num_compres**, **tiquet_mig**, **max_compra**, **total_gastat**.

```
1 WITH VIP_Stats AS (  
2 SELECT user_id,  
3 ROUND(SUM(amount), 2) as total_gastat,  
4 COUNT(*) as num_compres,  
5 ROUND(AVG(amount), 2) as tiquet_mig,  
6 MAX(amount) as max_compra  
7 FROM  
8 `sprint3_silver.transactions_clean`  
9 WHERE declined = 0  
10 GROUP BY user_id  
11 HAVING total_gastat > 500  
12 )  
13  
14 SELECT uc.user_id, uc.name as nom, uc.surname as cognom,  
15 uc.email, num_compres, tiquet_mig, max_compra, total_gastat  
16 FROM VIP_Stats  
17 JOIN `sprint3_silver.users_combined` AS uc  
18 ON VIP_Stats.user_id = uc.user_id  
19 ORDER BY total_gastat DESC  
20 ;
```

✓ Esta consulta procesará 2.55 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Recientes

▼ 11:47 a.m.

✓ WITH VIP_Stats AS (SELECT user_id, ROUND(SUM(amount), 2) as total_gastat, COUNT(*) as num_c...

- Ordenat per **total_gastat** descendent.

Exercici 2: Anàlisi de Tendències (Window Functions sobre Vistes)

Escenari:

La Direcció Financera vol monitoritzar la "velocitat de vendes" diària. Necessiten un informe que compari el rendiment de cada dia contra el dia anterior per detectar caigudes brusques o pics de creixement (Day-over-Day Growth)

Crea una consulta sobre la vista materialitzada que generi un informe amb les següents 4 columnes:

- » 1.**Data**: La data de venda.
- » 2.**Vendes_Avui**: El total venut aquell dia.
- » 3.**Vendes_Ahir**: El total venut el dia anterior (usant funcions de finestra).
- » 4.**Diff_Percentual**: El percentatge de creixement o decreixement respecte al dia anterior, arrodonit a 2 decimals.

💡 Pista Tècnica:

Utilitza la funció LAG(camp) OVER (ORDER BY data) per llegir el valor de la fila anterior sense necessitat de fer JOINS complexos.

```
1 WITH tendencia_ventas AS (
2   SELECT
3     day,
4     total_sales,
5     LAG(total_sales) OVER (ORDER BY day) AS sales_yesterday --para no repetir tres veces LAG...
6   FROM `sprint3_gold.mv_daily_sales`
7 )
8
9 SELECT
10  day AS Data,
11  total_sales AS Vendes_Avui,
12  sales_yesterday AS Vendes_Ahir,
13  ROUND(
14    ((total_sales - sales_yesterday) / sales_yesterday) * 100,
15    2
16  ) AS Diff_Percentual
17 FROM tendencia_ventas
18 ORDER BY day;
```

🟢 Esta consulta procesará 512 B cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Recientes	Resultados de...	Guardar los resultados	Abrir e
▼ 1:08 p.m.	< Información del trabajo	Resultados	Visualización
✓ WITH tendencia_ventas AS (...)	Hora de finalización	9 sept 2026, 1:08:40 p.m. UTC+2	JSC
▼ 1:01 p.m.	Duración	668 ms	
✓ SELECT day as Data, total_s...	Bytes procesados	512 B	
▼ 12:55 p.m.	Bytes facturados	10 MB	

Exercici 3: Totals Acumulats (Running Totals sobre Vistes)

Escenari:

El Director Financer (CFO) necessita visualitzar la corba de creixement anual. No li interessa tant el que es va vendre avui, sinó quant portem facturat en total des de l'inici de l'any fins avui (YTD - Year To Date).

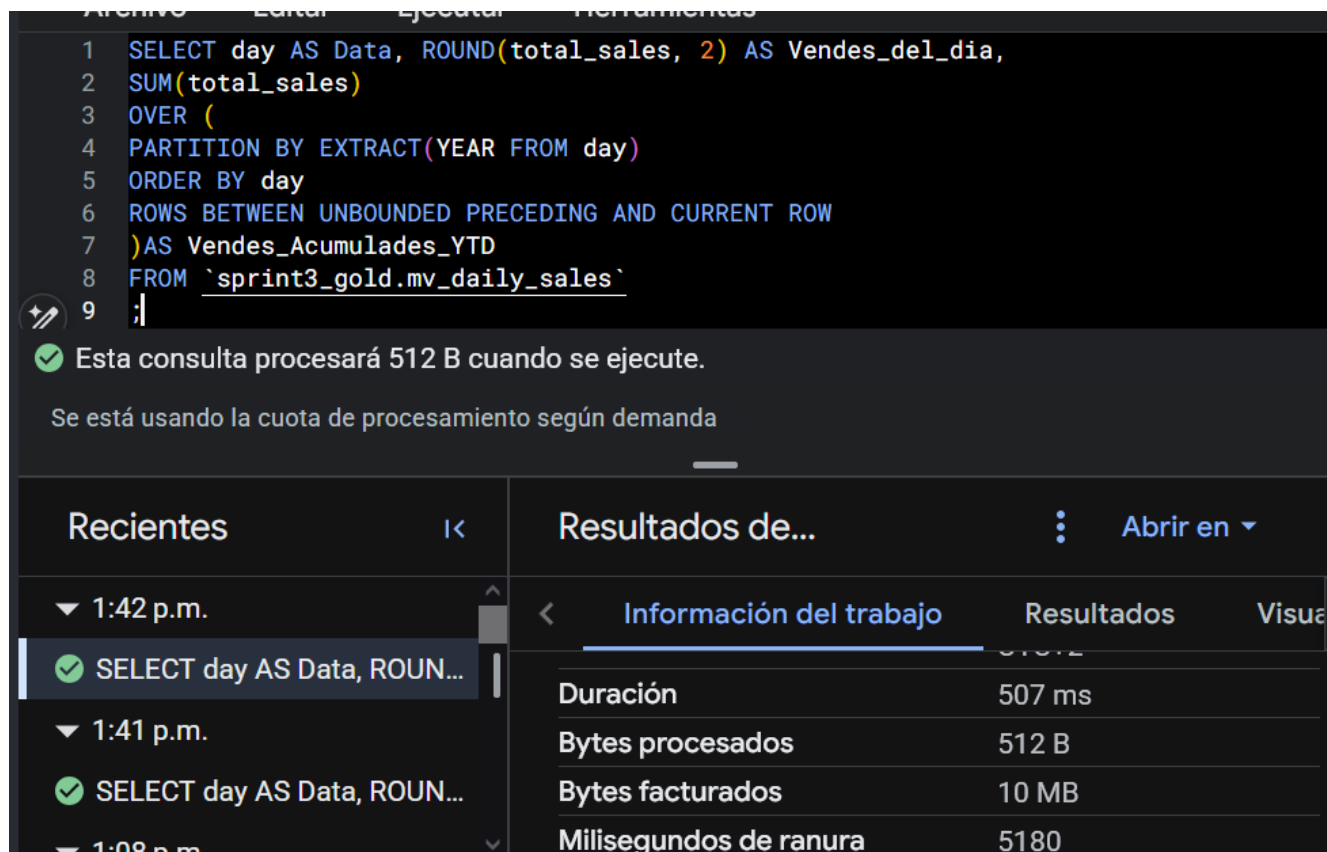
Tasca:

Utilitzant la vista materialitzada mv_daily_sales (per no recalculer agregacions base), genera un informe amb tres columnes:

- » 1.**Data**.
- » 2.**Vendes del Dia**: Arrodonit a 2 decimals.
- » 3.**Vendes Acumulades YTD**: Una suma progressiva de les vendes que ha de reiniciar-se cada 1 de Gener. El resultat final ha d'estar arrodonit a 2 decimals.

Pista Tècnica:

Per aconseguir el reinici anual, necessites dividir la finestra usant PARTITION BY EXTRACT(YEAR FROM data). L'estructura completa seria: SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).



```
1 SELECT day AS Data, ROUND(total_sales, 2) AS Vendes_del_dia,
2 SUM(total_sales)
3 OVER (
4 PARTITION BY EXTRACT(YEAR FROM day)
5 ORDER BY day
6 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
7 )AS Vendes_Acumulades_YTD
8 FROM `sprint3_gold.mv_daily_sales`
9 ;
```

✓ Esta consulta procesará 512 B cuando se ejecute.
Se está usando la cuota de procesamiento según demanda

Recientes	Resultados de...
1:42 p.m.	Información del trabajo
✓ SELECT day AS Data, ROUN...	Resultados
1:41 p.m.	Visual
✓ SELECT day AS Data, ROUN...	
1:08 p.m.	

Duración	507 ms
Bytes procesados	512 B
Bytes facturados	10 MB
Milisegundos de ranura	5180

Exercici 4: Fidelització i Valor del Client (Filtratge Avançat)

Escenari:

L'equip de Màrqueting llança la campanya "A la tercera va la vençuda". Volen enviar un regal als usuaris que hagin completat la seva tercera transacció. No obstant això, el pressupost del regal no és fix: depèn de la "qualitat" del client en els seus inicis. Necessiten saber el Tiquet Mitjà d'aquestes 3 primeres compres per segmentar si envien un detall bàsic o un regal premium.

Tasca:

Genera un llistat dels usuaris que han arribat (o superat) la seva tercera compra, mostrant:

- » 1. Dades d'usuari (user_id, nom_complet, email).
- » 2. La data i l'import exacte de la 3a compra.
- » 3. Mitjana de les 3 primeres: La mitjana de despesa de les seves transaccions 1, 2 i 3.

```
1  -- CTE 1 -> numerar compres
2  WITH TC1 AS (
3      SELECT ROW_NUMBER() OVER (PARTITION BY tc.user_id ORDER BY tc.timestamp) AS rn,
4             tc.user_id, uc.name, uc.surname, uc.email, tc.amount, tc.timestamp
5      FROM `sprint3_silver.transactions_clean` AS tc
6      JOIN `sprint3_silver.users_combined` AS uc
7      ON uc.user_id = tc.user_id
8      WHERE declined = 0
9      QUALIFY rn <= 3
10 )
11
12  -- CTE 2 -> media de las primeras 3
13  , TC2 AS (
14      SELECT user_id, round(AVG(amount), 2) AS avg
15      FROM TC1
16      GROUP BY user_id
17  )
18
19  -- SELECT final -> juntar todo
20  SELECT TC1.user_id, name, surname, email, amount AS importe_3a_compra,
21         DATE(timestamp) AS fecha_3a_compra,
22         TC2.avg AS avg_first_3
23  FROM TC1
24  JOIN TC2 ON TC1.user_id = TC2.user_id
25  WHERE rn = 3
26  ;
```

 Esta consulta procesará 3.31 MB cuando se ejecute.

Nivell 3: Analytics Engineering (Arrays & Automatització)

Escenari:

Màrqueting necessita urgentment un Rànquing de Productes més Venuts.

Exercici 1: Desanidament i Aplanament de Dades (Unnesting)

Escenari:

Actualment, a la taula de transaccions, els productes venuts estan "amagats" dins d'un Array (una llista d'IDs). L'equip de vendes vol analitzar quins productes específics es venen més, però no poden fer-ho si les dades estan comprimides en una llista. Necessiten una Taula Detallada (Flat Table) on cada fila representi un producte venut, encara que això signifiqui que l'ID de la transacció es repeteixi.

Tasca:

Crea la taula `dim_transactions_flat` desnormalitzant la informació. Has d'"explotar" l'array de productes i creuar-lo amb el catàleg mestre per obtenir els noms i preus individuals.

Passos Tècnics:

- » 1. Utilitza `CROSS JOIN UNNEST(product_ids)` per transformar l'Array en files individuals.
- » 2. Realitza un `JOIN` amb la taula `products` per obtenir el nom (`name`) i preu (`price`) de cada ítem.

 *Nota: Ves amb compte amb els tipus de dades en fer el JOIN (Array de INT64 vs Product_ID STRING).*

- » 3. No agrupeu el resultat. Volem veure el desglossament línia per línia.

Resultat Esperat (Visualització):

La teva taula final ha de veure's "plana", on una transacció amb 3 productes ocupa 3 files:

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_A	Portàtil Bàsic	99.99
TXN1001	2025-12-08	155.97	SKU_B	Ratolí	12.99
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclat	45.50

Entrega:

- » Una captura de la Vista Prèvia de la teva taula on es vegi el DDL executada correctament.
- » Una captura de pantalla on es vegi l'esquema de la nova taula.

```

1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat`
2 AS
3 SELECT transaction_id, DATE(timestamp) AS timestamp, amount AS total_ticket_GLOBAL,
4 product_id, pr.name AS product_name, pr.price AS product_price
5 FROM `sprint3_silver.transactions_clean` AS tr
6 CROSS JOIN UNNEST(tr.product_ids) AS product_id
7 JOIN `sprint3_silver.products_clean` AS pr
8 ON product_id = pr.product_id
9 WHERE tr.declined = 0
10 ORDER BY transaction_id;

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Recientes



Resultados de...



Abrir e...

▼ 11:01 a.m.

✓ CREATE OR REPLAC...

▼ 10:52 a.m.

✓ SELECT transaction_...



Información del trabajo

Resultados

Detalles d...



Esta declaración creó una nueva tabla con el nombre dim_transactions_flat.

[Ir a la tabla](#)



Nombre del campo

Tipo

Modo

Descripción



transaction_id

STRING

NULLABLE

-



timestamp

DATE

NULLABLE

-



total_ticket_GLOBAL

FLOAT

NULLABLE

-



product_id

INTEGER

NULLABLE

-



product_name

STRING

NULLABLE

-



product_price

FLOAT

NULLABLE

-



Exercici 2: El Rànquing de Vendes (Agregació Simple)

Escenari:

Ara que has preparat la infraestructura i disposes de la taula `dim_transactions_flat` (on cada fila equival a un producte venut), generar informes analítics és trivial. Ja no necessites funcions complexes de desanidament per respondre preguntes bàsiques de negoci.

✂ Tasca:

Genera el **Top 5 de productes** més venuts en la història de la companyia.

✂ Mètode:

Consulta directament la teva nova taula desnormalitzada (`dim_transactions_flat`).

- » En ser la taula plana, simplement necessites una agrupació estàndard (GROUP BY) pel nom del producte i un compte (COUNT) de les files.

👉 Entrega:

- » El codi SQL que genera el rànquing.
- » Una captura del resultat amb els 5 productes guanyadors i quantes unitats s'han venut de cadascun.

```
1 SELECT product_id, product_name, COUNT(*) AS num_transactions
2 FROM `sprint3_gold.dim_transactions_flat`
3 GROUP BY product_id, product_name
4 ORDER BY num_transactions DESC
5 LIMIT 5;
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Recientes	Resultados de...				Abrir en
11:48 a.m.	<	Información del trabajo	Resultados	Visualización	JS
✓ SELECT product_id, ...	Fila	product_id	product_name	num_transactions	
11:44 a.m.	1	52	riverlands the duel	2642	
✓ SELECT product_id, ...	2	29	Tully maester Tarly	2627	
11:44 a.m.	3	21	duel Direwolf	2603	
✓ SELECT product_id, ...	4	16	the duel warden	2602	
	5	33	duel warden	2593	

Exercici 3: Automatització del Pipeline i Visualització

Escenari:

L'informe estàtic de l'exercici anterior ha agradat molt, però el Director de Màrqueting necessita professionalitzar-lo per a la presa de decisions diària. Té dos requeriments urgents:

- » **1. Lògica Centralitzada:** Volen veure el preu amb IVA (21%) desglossat. No obstant això, com que l'impost pot canviar per país o any, et prohibeixen posar el càlcul $\text{preu} * 1,21$ directament en el codi ("Hardcoded"). Has d'utilitzar una funció reutilitzable.
- » **2. Frescor de Dades:** El tauler ha d'estar actualitzat automàticament cada matí a les 07:00 AM, abans de la reunió diària de vendes.

Tasca:

Has d'evolucionar la teva taula `dim_transactions_flat` per incloure el càlcul d'impostos i automatitzar la seva regeneració.

Passos Tècnics:

- » **1. User Defined Functions (UDF):** Crea una funció SQL persistent anomenada `calculate_tax(amount)` que rebí un valor numèric i retorni el resultat aplicant el 21% d'impost.

```
1 CREATE OR REPLACE FUNCTION sprint3_gold.calculate_tax(product_price FLOAT64)
2 RETURNS FLOAT64
3 AS
4   (product_price * 0.21) + product_price
5 ;
```

✓ Se completó la consulta

Recientes	Resultados de...
12:57 p.m. ✓ CREATE OR REPL...	Información del trabajo Resultados Detalles de la e
12:41 p.m. ✓ CREATE OR REPL...	

Esta declaración reemplazó la función llamada `sprint3-analytics-eduvallejo.sprint3_gold.calculate_tax`.

[Ir a la función](#)

- » 2. Integració i Orquestració:
 - Modifica el codi de creació de la taula (de l'Exercici 1) perquè utilitzi la teva nova funció `calculate_tax` i generi una columna nova: `product_price_tax_inc`.

```
1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat`
2 AS
3 SELECT transaction_id, DATE(timestamp) AS timestamp, amount AS total_ticket_GLOBAL,
4 product_id, pr.name AS product_name, pr.price AS product_price,
5 sprint3_gold.calculate_tax(pr.price) AS product_price_tax_inc
6 FROM `sprint3_silver.transactions_clean` AS tr
7 CROSS JOIN UNNEST(tr.product_ids) AS product_id
8 JOIN `sprint3_silver.products_clean` AS pr
9 ON product_id = pr.product_id
10 WHERE tr.declined = 0
11 ORDER BY transaction_id;
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Recientes

Resultados de...

1:01 p.m.

Información del trabajo

Resultados

Detalles de la

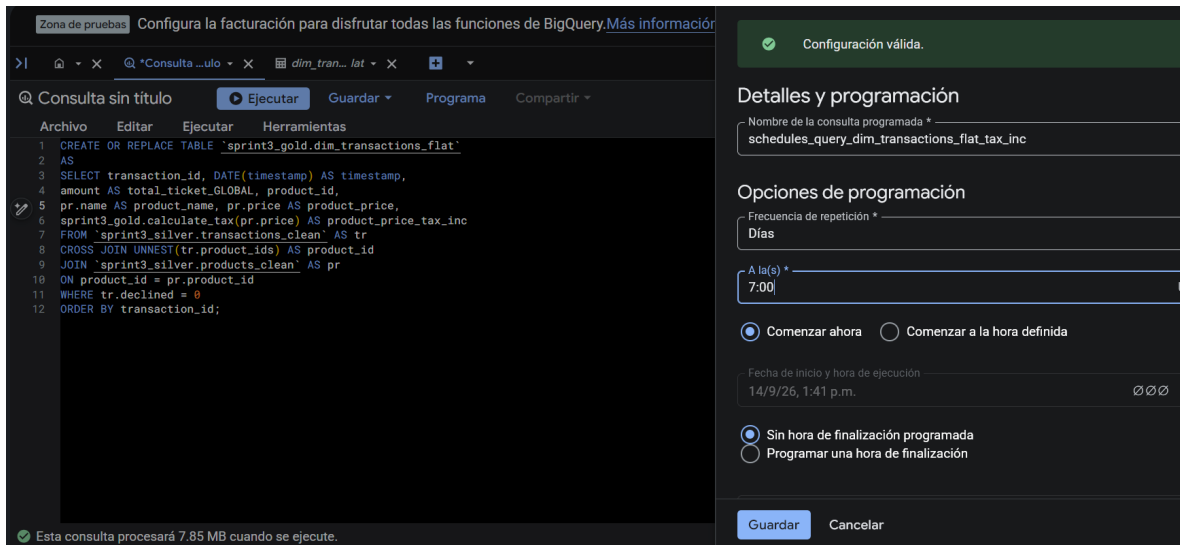
✓ CREATE OR REPLAC...

Hora de creación

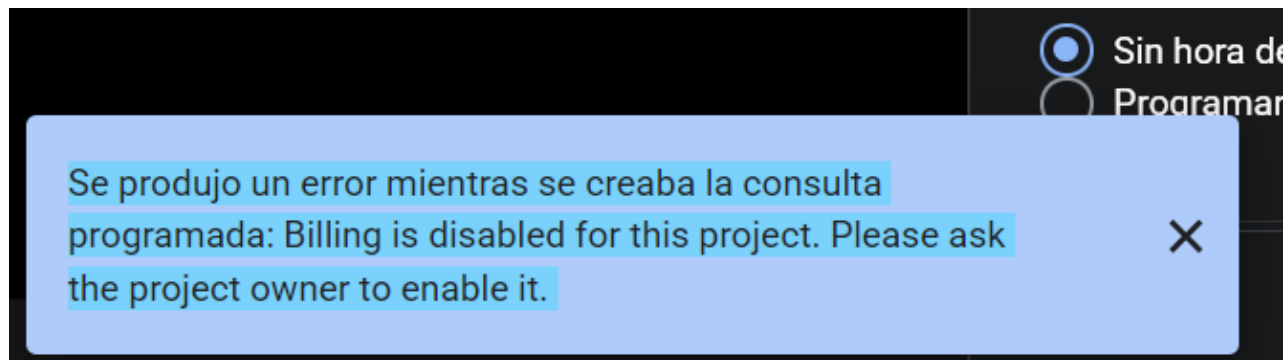
14 sept 2026, 1:01:22 p.m. UTC+2

☐	Nombre del campo	Tipo	Modo	Descri
☐	transaction_id	STRING	NULLABLE	-
☐	timestamp	DATE	NULLABLE	-
☐	total_ticket_GLOBAL	FLOAT	NULLABLE	-
☐	product_id	INTEGER	NULLABLE	-
☐	product_name	STRING	NULLABLE	-
☐	product_price	FLOAT	NULLABLE	-
☐	product_price_tax_inc	FLOAT	NULLABLE	-

- Configura una Scheduled Query a BigQuery perquè aquesta sentència (CTAS) s'executi automàticament cada dia a les 07:00 AM.



•



***Sin un plan de facturación no se pueden programar consultas**

- » **3. Visualització (BI):** Connecta Looker Studio a la teva taula dim_transactions_flat i crea un Dashboard: "Monitor de Rendiment de Vendes"

Enllaç a Looker Studio Report

<https://datastudio.google.com/embed/reporting/d292e006-e915-4f10-a468-911ab9fc35b6/page/uCr8F>